SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

CO1-ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Akash J	**Reg. No.:**	192324295
Section / Batch:	2023	**Date:**	15 - 10 - 2026

Code of Conduct

*I **Akash J / 192324295** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **Akash J**

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.
Identify email addresses.
Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Section 1: Regular Expressions – Resume Information Extraction

1. Aim

To design and implement a Python-based resume information extraction system using Regular Expressions.

2. Algorithm

1. Import the re module.
2. Store resume details as text.
3. Extract the candidate's name, email, and mobile number using regular expressions.
4. Identify technical skills such as Python, Java, SQL, Machine Learning, and NLP.
5. Extract years of experience.
6. Generate a structured profile summary.
7. Display candidates who have at least 2 years of experience and Python skill.

3. Program

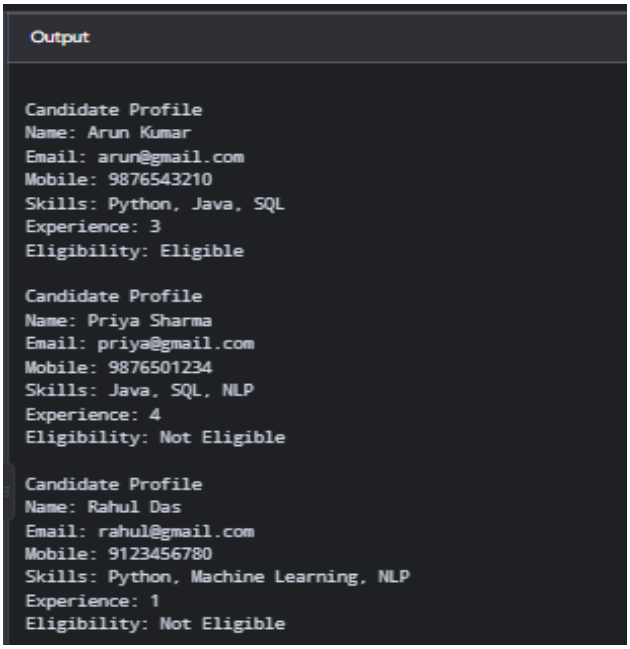
```
import re

resumes = [
    """
    Name: Arun Kumar
    Email: arun@gmail.com
    Mobile: 9876543210
    Skills: Python, Java, SQL
    Experience: 3 years
    """,
    """
    Name: Priya Sharma
    Email: priya@gmail.com
    Mobile: 9876501234
    Skills: Java, SQL, NLP
    Experience: 4 years
    Name: Rahul Das
    Email: rahul@gmail.com
    Mobile: 9123456780
```

```
    Skills: Python, Machine Learning, NLP
    Experience: 1 year
    """
]
for resume in resumes:
    name = re.search(r"Name:\s*(.+)", resume)
    email = re.search(
        r"[\w.-]+@[\w.-]+\.\w+", resume
    )
    mobile = re.search(
        r"Mobile:\s*(\+?\d[\d\s-]{8,}\d)", resume
    )
    exp = re.search(
        r"Experience:\s*(\d+)\s*years?", resume,
        re.IGNORECASE
    )
    skills_list = [
        "Python", "Java", "SQL",
        "Machine Learning", "NLP"
    ]
    skills = [
        skill for skill in skills_list
        if re.search(
            r"\b" + re.escape(skill) + r"\b",
            resume, re.IGNORECASE
        )
    ]
    years = int(exp.group(1)) if exp else 0
    profile = {
        "Name": name.group(1).strip() if name else "N/A",
        "Email": email.group() if email else "N/A",
        "Mobile": mobile.group(1).strip() if mobile else "N/A",
```

```
        "Skills": ", ".join(skills) if skills else "None",
        "Experience": years
    }
    print("\nCandidate Profile")
    for key, value in profile.items():
        print(f"{key}: {value}")
    if years >= 2 and "Python" in skills:
        print("Eligibility: Eligible")
    else:
        print("Eligibility: Not Eligible")
```

4. Output



5. Result

The Python program successfully extracts candidate information using regular expressions, generates structured profile summaries, and identifies eligible candidates based on the minimum requirements of 2 years of experience and Python skill.

Section 2: Regular Expressions – Product Search System

1. Aim

To design and implement a Python-based product search system using Regular Expressions to perform exact, prefix, suffix, partial, and case-insensitive searches and generate a report of matching products.

2. Algorithm

1. Create a list of product names.
2. Import the re module.
3. Accept a search keyword and search type.
4. Use regular expressions to find matching products.
5. Display all matching product names.
6. Count the matching products for each search.
7. Generate a search report showing the total matches.

3. Program

```
import re

products = [
    "Apple iPhone",
    "Samsung Galaxy",
    "Apple MacBook",
    "Dell Laptop",
    "Wireless Mouse",
    "Bluetooth Headphones",
    "Apple Watch",
    "Gaming Laptop"
]

def search_products(keyword, search_type):
    if search_type == "exact":
        pattern = r"\b" + re.escape(keyword) + r"\b"
```

```
elif search_type == "prefix":
    pattern = r"\b" + re.escape(keyword)
elif search_type == "suffix":
    pattern = re.escape(keyword) + r"\b"
elif search_type == "partial":
    pattern = re.escape(keyword)
elif search_type == "case-insensitive":
    pattern = re.escape(keyword)
else:
    print("Invalid search type")
    return
flags = re.IGNORECASE if search_type == "case-insensitive" else 0
matches = [
    product for product in products
    if re.search(pattern, product, flags)
]
print(f"\nSearch Type: {search_type}")
print(f"Keyword: {keyword}")
if matches:
    print("Matching Products:")
    for product in matches:
        print("-", product)
else:
    print("No matching products found.")
print("Total Matches:", len(matches))
```

```
    return len(matches)

# Perform different searches

report = {}

report["Exact"] = search_products("Apple", "exact")

report["Prefix"] = search_products("Apple", "prefix")

report["Suffix"] = search_products("Laptop", "suffix")

report["Partial"] = search_products("top", "partial")

report["Case-insensitive"] = search_products(

    "apple", "case-insensitive"

)

print("\n----- SEARCH REPORT -----")

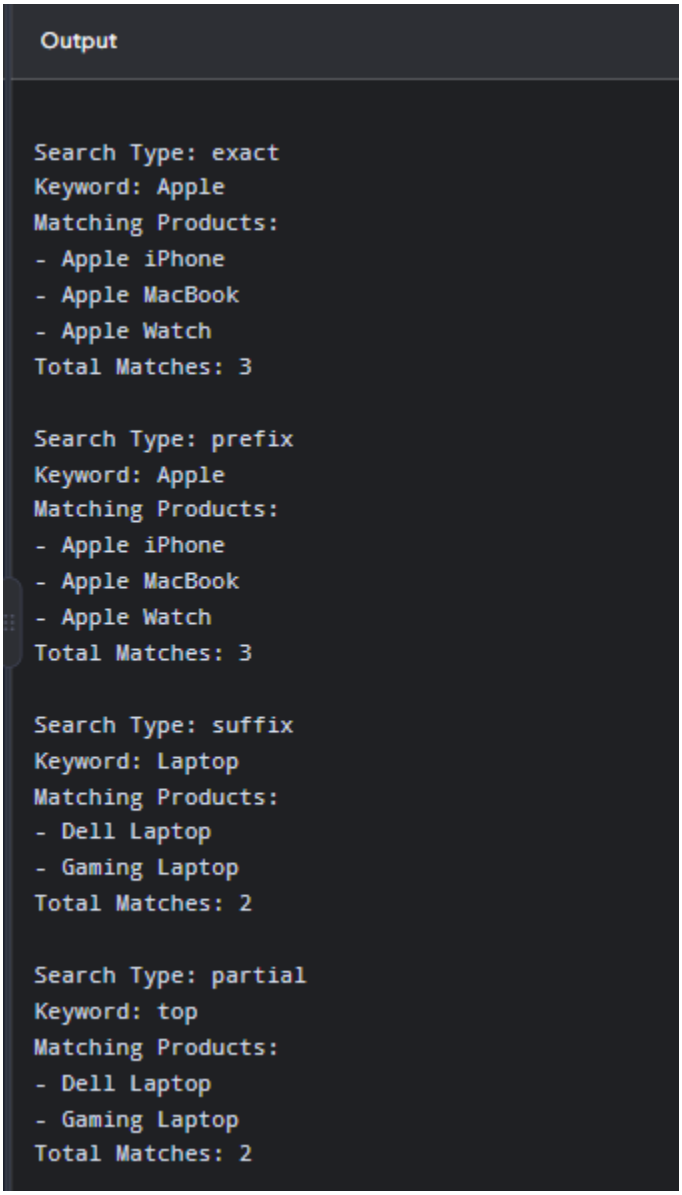
for search_type, count in report.items():

    print(f"{search_type}: {count} matching products")
```

4. Output

```
Search Type: case-insensitive
Keyword: apple
Matching Products:
- Apple iPhone
- Apple MacBook
- Apple Watch
Total Matches: 3

----- SEARCH REPORT -----
Exact: 3 matching products
Prefix: 3 matching products
Suffix: 2 matching products
Partial: 2 matching products
Case-insensitive: 3 matching products
```



5. Result

The program successfully performs exact, prefix-based, suffix-based, partial keyword, and case-insensitive product searches using Regular Expressions. It displays all matching product names and generates a report containing the total number of matches for each search.

Section 3: Student Registration Validation Using Regular Expressions

1. Aim

To design and implement a Python-based student registration validation system using Regular Expressions to validate student details and generate a final registration status report.

2. Algorithm

1. Import the re module.
2. Get the student's register number, institutional email, course code, semester, and mobile number.
3. Validate each field using Regular Expressions.
4. Display a suitable validation message for each field.
5. Check whether all fields are valid.
6. Display the final registration status report.

3. Program

```
import re

# Student details
reg_no = "23CS101"
email = "student@university.edu"
course = "CS101"
semester = "5"
mobile = "9876543210"

# Validation patterns
patterns = {
    "Register Number": r"^\d{2}[A-Z]{2}\d{3}$",
    "Institutional Email": r"^[\w.-]+@university\.edu$",
    "Course Code": r"^[A-Z]{2}\d{3}$",
    "Semester": r"^[1-8]$",
    "Mobile Number": r"^[6-9]\d{9}$"
}

# Student data
details = {
    "Register Number": reg_no,
    "Institutional Email": email,
    "Course Code": course,
    "Semester": semester,
    "Mobile Number": mobile
```

```
}

all_valid = True

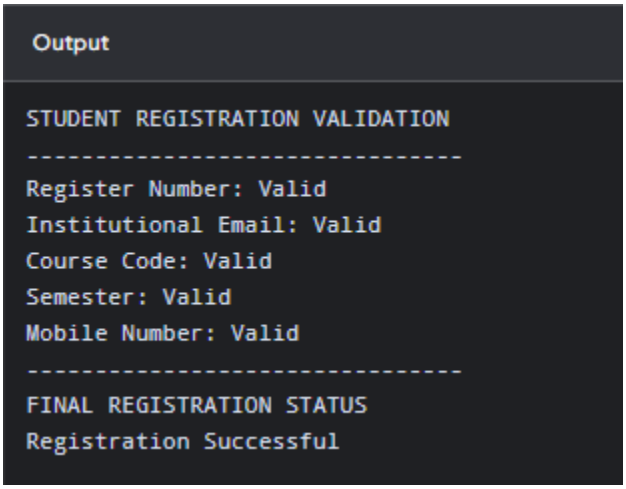
print("STUDENT REGISTRATION VALIDATION")
print("--------------------------------")

for field, value in details.items():
    if re.fullmatch(patterns[field], value):
        print(field + ": Valid")
    else:
        print(field + ": Invalid")
        all_valid = False

print("--------------------------------")
print("FINAL REGISTRATION STATUS")

if all_valid:
    print("Registration Successful")
else:
    print("Registration Failed")
    print("Please correct the invalid details.")
```

4. Output



5. Result

The Python program successfully validates the student's register number, institutional email, course code, semester, and mobile number using Regular Expressions. It displays individual validation messages and generates a final report indicating whether the registration is successful.